

Deepfake Detection System: Thesis & Presentation Guide

This document is specifically tailored for your **FYP Presentation preparation** and your **Thesis write-up** (Methodology and Results sections). It breaks down the code file-by-file, explains the technical pipeline with diagrams, and discusses the model's accuracy.

1. FYP Presentation: File-by-File Code Breakdown

For your presentation, you need to explain how the code works logically. The project is divided into **four main components**:

app.py (The API / Gateway)

- **Purpose:** This is the entry point of the system. It creates a web server that mobile apps or websites can communicate with.
- **Key Operations:**
 1. **Initialization (`initialize_model`):** Loads the heavy AI model into memory (RAM/VRAM) exactly once when the server starts, preventing delays during requests.
 2. **Receiving Data (`analyze_video`):** Receives the .mp4 file sent by the user, saves it temporarily, and performs security checks (file type, size limit).
 3. **Bridging:** Hands the saved video path to `inference.py` for processing, waits for the result, and returns a structured JSON response (Fake/Real + confidence percentages).

inference.py (The Data Processing Engine)

- **Purpose:** Bridges the gap between a raw video file and the AI model. It contains the logic to convert a video into mathematical tensors.
- **Key Operations (`VideoProcessor` Class):**
 1. **Frame Extraction (`extract_frames`):** Calculates the video length and extracts exactly **20 evenly spaced frames**. This ensures the model sees the beginning, middle, and end of the video regardless of its total length.
 2. **Face Detection & Cropping:** Uses the `face_recognition` library (powered by `dlib`) to locate the human face in each frame. It crops the image tightly around the face with a 40-pixel padding. *Why? Because deepfake artifacts are exclusively on the face; analyzing the background introduces noise.*
 3. **Preprocessing (`preprocess_frame`):** Resizes the cropped face to 112×112 pixels and normalizes the colors (matching the ImageNet dataset standards) so the AI model can digest it.

model.py (The Brain / AI Architecture)

- **Purpose:** Defines the deep learning neural network structure.
- **Key Operations (`DeepFakeDetector` Class):**

1. **Spatial Feature Extraction:** Uses a **ResNeXt50** backbone. It looks at each of the 20 frames individually to find spatial inconsistencies (e.g., blurry teeth, mismatched skin tones, warped edges).
2. **Temporal Feature Extraction:** Uses an **LSTM** (Long Short-Term Memory) network. It looks at the *sequence* of the 20 frames to find temporal inconsistencies (e.g., unnatural blinking, jittering, inconsistent head movement over time).
3. **Classification:** Passes the final data through a Linear layer with Dropout (to prevent overfitting) to output the final probabilities for "REAL" vs "FAKE".

requirements.txt (The Environment)

- **Purpose:** The recipe book for the project. It lists exact versions of libraries (`torch`, `dlib`, `opencv-python`) required to recreate your exact working environment so the project doesn't break on another computer.

2. Thesis: Methodology Section

You can use the following text and diagrams directly in your thesis Methodology section.

2.1 System Pipeline Overview

The proposed deepfake detection system operates through a sequential pipeline designed to isolate facial features and analyze spatial-temporal inconsistencies.

flowchart TD

```

A[Raw Video Upload] --> B[Frame Extraction]
B -->|Select 20 equidistant frames| C[Face Detection & Cropping]
C -->|dlib CNN| D[Preprocessing]
D -->|Resize 112x112, Normalize| E[Tensor Sequence Creation]
E -->|Shape: 1 x 20 x 3 x 112 x 112| F[ResNeXt-50 Backbone]
F -->|Spatial Features: 2048-dim| G[LSTM Network]
G -->|Temporal Modeling| H[Fully Connected Layer]
H --> I[Softmax Output]
I -->|0| J[FAKE]
I -->|1| K[REAL]

```

2.2 Face Extraction and Preprocessing

Not all parts of a video frame are relevant for deepfake detection; manipulations are heavily localized to the facial region. The methodology employs the `dlib` library for robust facial landmark detection. For each selected frame, the bounding box of the face is identified, and a padding of 40 pixels is added to capture the structural contours of the head and chin, where blending artifacts frequently occur. If no face is detected in a specific frame, the system falls back to using the full frame to maintain sequence integrity. The cropped regions are then resized to 112×112 pixels and normalized using ImageNet mean $([0.485, 0.456, 0.406])$ and standard deviation $([0.229, 0.224, 0.225])$.

2.3 Proposed Deep Learning Architecture

The core architecture is a hybrid spatial-temporal neural network designed to capture both

frame-level artifacts and video-level temporal anomalies.

```
graph LR
    subgraph Spatial Analysis
        A[Frame 1] --> R1[ResNeXt-50]
        B[Frame 2] --> R2[ResNeXt-50]
        C[...] --> R3[...]
        D[Frame 20] --> R4[ResNeXt-50]
    end

    subgraph Temporal Analysis
        R1 --> L1[LSTM Cell 1]
        R2 --> L2[LSTM Cell 2]
        R3 --> L3[...]
        R4 --> L4[LSTM Cell 20]
        L1 --> L2
        L2 --> L3
        L3 --> L4
    end

    subgraph Classification
        L4 --> FC[Linear Layer]
        FC --> D[Dropout 0.4]
        D --> O[Softmax 2-Class]
    end
```

1. **Spatial Backbone (ResNeXt-50):** We utilize a ResNeXt50_32x4d network, pre-trained on ImageNet. The final average pooling and fully connected layers are removed. The model processes the spatial dimensions of the input tensor, extracting high-level semantic features (2048 dimensions per frame) that represent spatial manipulation artifacts like pixel blending errors or resolution mismatches.
 2. **Temporal Network (LSTM):** The sequence of 20 feature vectors is fed into a Long Short-Term Memory (LSTM) network with a hidden dimension of 2048. Deepfakes often struggle with temporal consistency (e.g., unnatural blinking patterns or jittering between frames). The LSTM retains the memory of previous frames to detect these temporal discrepancies.
 3. **Classifier:** The final hidden state of the LSTM is passed through a Dropout layer ($p=0.4$) for regularization, followed by a linear classification layer mapping to 2 output classes (REAL and FAKE).
-

3. Thesis: Results & Discussion

3.1 Model Accuracy Discussion

The model achieved an overall accuracy of **87%** on the testing dataset. This demonstrates strong predictive capability, especially considering the model relies on a lightweight sequence of only 20 frames resized to 112×112 pixels. It successfully identifies spatial artifacts introduced by generative models and temporal flickering across frame sequences.

3.2 Limitations: Why Not > 90% Accuracy?

While 87% is a robust result for standard deepfake detection, several inherent limitations in the current methodology prevent the system from achieving > 90% accuracy. You can present these in your thesis as **Limitations and Future Work**:

Limitation Factor	Explanation	Impact on Accuracy
Low Input Resolution	Faces are heavily downscaled to 112×112 pixels to save computational power and memory.	High-frequency spatial artifacts (micro-textures, tiny blending boundaries at the edge of a fake face) are destroyed during downscaling, causing the model to miss subtle fakes.
Fixed Sequence Length	The system extracts exactly 20 frames per video, regardless of whether the video is 3 seconds or 3 minutes long.	For longer videos, extracting only 20 frames means skipping large chunks of time. The model might miss the specific moments where the deepfake algorithm glitches.
Face Detection Failures	Deepfakes often feature extreme head angles or motion blur. If the <code>dlib</code> detector fails to find a face, the code defaults to processing the entire uncropped frame.	The model expects a zoomed-in face. When fed a full frame, the facial features become too small to analyze, leading to false negatives (classifying a fake as real).
Compression Artifacts	Videos downloaded from social media are heavily compressed (H.264/H.265).	Video compression introduces its own blocky artifacts which confuse the ResNeXt backbone. It becomes difficult for the model to distinguish between "compression noise" and "deepfake noise".
Generative AI Evolution	The model was trained on a specific dataset. Modern diffusion models (like Sora or Midjourney) create fakes with vastly different artifact patterns than older GAN-based methods.	Models suffer from <i>domain shift</i> . They perform well on the types of deepfakes they were trained on, but accuracy drops on unseen, state-of-the-art generation techniques.

3.3 Future Enhancements (To mention in conclusion)

To push the accuracy beyond 90% in future iterations, the following architectural upgrades could be explored:

- 1. Vision Transformers (ViT):** Replacing ResNeXt with a TimeSformer or Video Vision Transformer to better capture global context instead of relying purely on CNN convolutions.
- 2. Audio-Visual Modality:** Incorporating the audio track. Current deepfakes often have slight audio-lip desynchronization. Combining audio analysis with video analysis drastically increases detection accuracy.
- 3. Higher Resolution Processing:** Utilizing a 224×224 or 256×256 input resolution paired with a deeper network like EfficientNet to capture micro-texture manipulation.